Operational Protocol Upgrade: Transitioning Stela Prompts to Phase 3 Immutable Workflows

1. The Epistemological Crisis of Mutable Governance

The governance architecture of the Stela repository is predicated on a singular, inviolable concept defined in the Policy of Truth (PoT): the system prevents action that violates policy rather than requesting compliance.[1] This doctrine, formalized as "Phase 3: Immutable Workflow Adoption," represents a paradigm shift from narrative-driven file manipulation to deterministic state generation. The transition is not merely administrative; it is a response to the "entropy of prose" where human and agent operators, when left to interact with mutable control surfaces like TASK.md via unstructured text, inevitably introduce structural drift, ambiguity, and phantom canonical states.[1]

The user query necessitating the update of docs/ops/prompts/ to align with the ops/bin/draft workflow highlights a critical friction point in the previous operational phases. In Phase 1 and 2, the "Truth" of a Dispatch Packet (DP) was contained within the narrative body of TASK.md, a file subject to manual edits, copy-paste errors, and structural degradation.[1] The introduction of ops/bin/draft—a binary executable designed to mechanically render the DP from a strict set of inputs—demands that the upstream inputs (the Agent's output) be rigorously standardized.

This report provides an exhaustive technical specification for refactoring the operator prompt ecosystem (E-PROMPT-01 through 04) to interface with this new immutable machinery. The analysis synthesizes the constraints of the ops/bin/draft source code [1], the security requirements of the sidecar allowlist mechanism (storage/dp/active/allowlist.txt) [1], and the cognitive requirements of the "Strategic Options" menu to deliver a "Correct by Construction" (CbC) operational environment.

1.1 The Failure of Narrative Mutability

To understand the necessity of the prompt refactoring, one must first analyze the failure modes of the legacy workflow. In the previous model, an Architect Agent would output a Markdown block starting with ### DP-OPS-XXXX. The Operator was then responsible for manually splicing this block into TASK.md.[1] This process introduced three distinct vectors for "System Failure States" as defined in PoT.md [1]:

1. **Canon Drift via Structural Erosion:** LLMs, when generating Markdown, often hallucinate or omit distinct structural markers (e.g., changing ### to ## or renaming "Execution Plan" to "Plan of Execution"). These minor deviations break the regex-based

linters in tools/lint/dp.sh, which rely on exact string matching to validate the governance contract.[1]

2. **Dirty State Injection:** Manual pasting allows for the accidental inclusion of "disposable artifacts"—rough notes, chat logs, or unverified claims—into the canonical record. The PoT explicitly forbids this, stating that "Action: deception or incoherence are inherently destabilizing".[1]
3. **Ambiguity in File Access:** In narrative DPs, the "Allowlist" was often a bulleted list buried within Section 3.3. This made programmatic enforcement difficult; a linter would have to parse natural language to determine which files were legally modifiable.

Phase 3 resolves these issues by decoupling the *definition* of work from the *rendering* of the control surface. The "Truth" is no longer the text in TASK.md; the Truth is the "Slots File"—a structured configuration artifact—and the allowlist.txt sidecar. TASK.md becomes a read-only compilation target, generated deterministically by ops/bin/draft.

1.2 The Correct by Construction (CbC) Philosophy

The Policy of Truth Implementation Plan outlines the move toward CbC.[1] The core tenet is that it should be physically impossible to create a malformed Dispatch Packet. By mandating that Agents output a "Slots File" (a specific format consumed by the draft binary) rather than raw Markdown, we enforce CbC at the generation layer.

- **Legacy:** Agent generates Markdown -> Operator pastes -> Linter checks if it's broken (Post-hoc Validation).
- **Phase 3:** Agent generates Slots -> ops/bin/draft consumes Slots -> Binary generates valid Markdown (Correct by Construction).

The prompt updates detailed in this report are the "software update" for the Agents, enabling them to function as valid inputs for this new "hardware."

2. Technical Anatomy of the Phase 3 Toolchain

Before defining the specific prompt text, we must rigorously analyze the tools the prompts interact with. The requirements for E-PROMPT-03 (Architect) and E-PROMPT-02 (Hygiene) are dictated entirely by the input expectations of ops/bin/draft.

2.1 Dissecting ops/bin/draft

The source code for ops/bin/draft, located in the repository dump [1], reveals it is a Bash script that renders a template based on parsed "slots." The critical function governing input format is read_slot:

Bash

```
read_slot() {
  local key="$1"
  local file="$2"
  awk -v key="$key" '
    $0 == "[" key "]" { in_block=1; next }
    in_block && $0 ~ /^\[[A-Z0-9_]+\]$/ { exit }
    in_block { print }
  ' "$file"
}
```

Technical Implications for Prompt Engineering:

1. **Exact Header Syntax:** The awk script looks for lines exactly matching ``. It does not support Markdown headers like ## or ****. The prompt must strictly forbid Markdown formatting for the slot headers.
2. **Block Termination:** The parser reads until it sees another line starting with [and ending with]. If the Agent generates loose text that happens to look like a tag (e.g., "See"), it could prematurely terminate the block. The prompt must instruct the Agent to sanitize content to avoid false terminators.
3. **Variable Slots:** The script looks for specific keys: DP_SCOPED_LOAD_ORDER, OBJECTIVE, IN_SCOPE, OUT_OF_SCOPE, SAFETY_INVARIANTS, PLAN_STATE, PLAN_REQUEST, PLAN_CHANGELOG, PLAN_PATCH, and RECEIPT_COMMANDS.[1] The Agents must use these exact keys.

2.2 The Canonical Template (ops/src/surfaces/DP.md.tpl)

The draft binary populates ops/src/surfaces/DP.md.tpl.[1] This template contains immutable boilerplate that the Agent *cannot* change. For example, the section headers "3.1 Freshness Gate" or "3.3 Scope and Safety" are hardcoded in the template.

Constraint on E-PROMPT-03: The Architect Agent must be explicitly told *not* to generate section headers. If the Agent outputs:

3.3 Scope and Safety

Fix the bugs.

The draft binary will insert this into the template, resulting in duplicate headers:

3.3 Scope and Safety

3.3 Scope and Safety

Fix the bugs. This duplication would violate the TASK.md schema enforced by tools/lint/task.sh.[1] Thus, the prompt must enforce "Content Only" generation wrapped in the specific Slot tags.

2.3 The Sidecar Allowlist Mechanism

The user query highlights the enforcement of storage/dp/active/allowlist.txt. In previous phases, file permissions were defined in prose within TASK.md. Phase 3 moves this to a designated configuration file.

Security Architecture: The tools/lint/dp.sh script [1] validates that the DP contains a pointer to the allowlist: {{ALLOWLIST_POINTER}}. It then verifies that the file exists and contains valid paths.

- **Decoupling:** The "Intent" (what we want to do) is in the DP. The "Permission" (what we are allowed to touch) is in the sidecar.
- **Gatekeeper Responsibility:** The Gatekeeper (E-PROMPT-01) must now perform a cross-referencing operation. It must read the [PLAN_CHANGELOG] or `` from the DP and verify that every file mentioned there is present in storage/dp/active/allowlist.txt.

3. The Architecture of the Slots File

The "Slots File" is the new intermediate artifact in the Stela workflow. It serves as the bridge between the unstructured reasoning of the AI Agent and the structured execution of the draft binary. The user requirement specifying "backticked file paths instead of raw Markdown" is a critical syntactic constraint that enhances machine readability and prevents hallucination.

3.1 The Necessity of Backticked Paths

In raw Markdown, a file path can be ambiguous. Is README.md a file path, or is it the subject of a sentence? By enclosing all paths in backticks (`ops/bin/draft`), we create a distinct token that is:

1. **Visually Explicit:** The Operator can immediately scan the Slots File and see exactly which files are being targeted.
2. **Machine Parseable:** Future iterations of draft or lint scripts can easily regex extract contents between backticks to validate existence or permissions.
3. **Formatting Safe:** Characters like underscores in filenames (e.g., opt/_factory/INDEX.md) can trigger Markdown italics if not escaped or backticked. Backticks ensure the path is treated as a literal string.

3.2 Slot Definitions and Content Requirements

Based on the ops/bin/draft script [1], the Slots File is not a JSON or YAML file, but a custom tag-delimited text file. The prompt must define the expected content for each slot:

Table 1: Phase 3 Slot Definitions for Agent Prompts

Slot Key	Content Requirement	Prompt Instruction
``	List of paths.	Must include relevant tools (e.g., `tools/lint/dp.sh`).
``	Narrative text.	Concise, one-sentence goal.
``	Bulleted list.	Explicit definition of work boundaries.
``	Bulleted list.	Explicit definition of what is *not* happening.
``	Bulleted list.	Must include "Do not hand-edit generated outputs."
``	Narrative/List.	Current status derived from OPEN analysis.
``	Narrative.	Translation of the Operator's intent.
[PLAN_CHANGELOG]	List of files.	**CRITICAL:** Used to populate allowlist sidecar.
``	Numbered steps.	The detailed execution logic.
``	Bash commands.	Must include lint/truth.sh, lint/dp.sh, verify.sh.

4. Refactoring E-PROMPT-03 (Architect)

The Architect prompt is the primary engine for generating new work. In Phase 3, it transforms from a "Writer" to a "Compiler."

4.1 From Narrative to Configuration

The previous E-PROMPT-03 instructed the Agent to "DRAFT ... matching TASK.md exactly".[1] This often led to the Agent attempting to reproduce the entire TASK.md file structure, including sections that should remain static.

The New Directive:

The prompt must explicitly forbid the generation of the DP document. Instead, it must command the generation of the *inputs* for the draft tool.

Specific Requirement: Backticked Paths:

The prompt must include a constraint: "All file path references within the Slots output must be enclosed in backticks." This ensures that when the draft binary renders the template, the resulting TASK.md contains properly formatted paths, and the intermediate Slots File is safe from Markdown rendering issues.

4.2 Updated Content Specification

The prompt will be structured to guide the Agent through:

1. **Analysis:** Reading the plan.md and the current state.
2. **Mapping:** Converting the plan into the 10 specific slots.
3. **Formatting:** Applying the bracketed header syntax and backticked paths.

(See Section 9 for the full text of the updated prompt).

5. Refactoring E-PROMPT-02 (Hygiene)

The Hygiene prompt serves as the "Sanitizer." Its role is to take a malformed or legacy DP (e.g., one written in Phase 2 style) and convert it into the Phase 3 standard.

5.1 The Normalization Function

In the immutable workflow, Hygiene becomes critical for "Intake." If an Operator has a rough idea scribbled in a text file, Hygiene normalizes it into a Slots File.

Key Updates:

- **Input:** Accepts Old-DP.md or rough-notes.txt.
- **Output:** ops/bin/draft Slots File.
- **Constraint:** Like the Architect, it must enforce backticked paths. It must also ensure that "disposable artifacts" (like chat citations) are stripped out, as TASK.md must contain only canon pointers.[1]

5.2 Bridging the Gap

Hygiene is the bridge between "Human Thought" (messy, unstructured) and "System Execution" (rigid, structured). The prompt must emphasize *extraction* of intent over *reproduction* of text. It must read the old DP, understand what it *meant* to do, and rephrase that intent into the valid and slots.

(See Section 9 for the full text of the updated prompt).

6. Refactoring E-PROMPT-04 (Analyst)

The Analyst prompt currently outputs "Discussion only".[1] This open-ended instruction often leads to "Analysis Paralysis," where the Agent provides a wall of text without a clear path forward. The user requirement for a "Strategic Options" menu is a direct countermeasure to this cognitive friction.

6.1 The "Strategic Options" Menu

This pattern forces the Analyst to synthesize its findings into discrete, executable choices. It aligns with the "Linguistic Precision" mandate of the PoT [1] by removing ambiguity about the next steps.

Structure of the Menu:

- **Option 1:**
  - *Pros:* Why do this?
  - *Cons:* What is the cost?
  - *Risk:* What could go wrong?
- **Recommendation:** A clear directive on which option aligns best with the PoT.

6.2 Minimizing Operator Prose

By presenting options (1, 2, 3), the Agent enables the Operator to respond with a single character ("1"). This drastically reduces the "Operator prose" required to drive the session, fulfilling the specific goal of the user query. It shifts the interaction model from "Collaborative Chat" to "Executive Review."

(See Section 9 for the full text of the updated prompt).

7. Refactoring E-PROMPT-01 (Gatekeeper)

The Gatekeeper is the final line of defense against drift. In Phase 3, its auditing scope expands to include the sidecar allowlist.

7.1 Enforcing the Sidecar

The Gatekeeper must now look beyond TASK.md. It must verify the existence and content of storage/dp/active/allowlist.txt.

The Audit Logic:

1. **Existence:** Does storage/dp/active/allowlist.txt exist? (If no -> FAIL).
2. **Content:** Does the allowlist contain every file modified in the Pull Request? (If no -> FAIL).
3. **Hazards:** Does the allowlist contain forbidden paths like opt/_factory without explicit justification? (If yes -> FAIL).

7.2 Slots Integrity (Implicit)

While the Gatekeeper primarily audits the result (TASK.md and code changes), it implicitly audits the Slots File by verifying that the TASK.md structure is correct. If the structure is broken (e.g., duplicate headers), it implies a failure in the drafting process, which the Gatekeeper must flag.

(See Section 9 for the full text of the updated prompt).

8. Updating the README

The docs/ops/prompts/README.md serves as the "Instruction Manual" for the prompt system. It must be updated to explicitly mandate the new workflow. The "Global Rules" must now include the prohibition of manual TASK.md edits and the requirement to use ops/bin/draft.

9. Comprehensive Implementation Specifications

The following subsections provide the exact, immutable text for the updated files. These specifications integrate all requirements: backticked paths, slots file format, strategic options, and sidecar enforcement.

9.1 docs/ops/prompts/E-PROMPT-03.md (Architect)

Architect (Refresh + Generate Slots)

Use when: Drafting a new DP configuration for the ops/bin/draft workflow.

Attach: OPEN, OPEN-PORCELAIN, dump, dump manifest, plan.md.

Rules:

- Refresh state using the attached OPEN and dump artifacts.
- Logic: PoT.md. Structure: ops/bin/draft Slots Format. Output only the stance format.
- Do not invent file paths; verify with the dump.
- **Phase 3 Mandate:** Output must be a valid Slots File compatible with ops/bin/draft.
- **Path Syntax:** All file paths must be enclosed in backticks (e.g., ops/bin/draft).

Steps:

1. ANALYZE <plan.md> to determine Objective, Scope, and Execution Plan.
2. CONSTRUCT a Slots File using the specific headers required by ops/bin/draft:
   - ``: List specific lints and specs required.
   - ``: Concise statement of goal.
   - ``: Bulleted list of work.
   - ``: Explicit exclusions (context hazards).
   - ``: "Do not hand-edit generated outputs", etc.
   - ``: Current context derived from OPEN.
   - ``: Translation of plan requirements.
   - [PLAN_CHANGELOG]: List of files to be created/modified.
   - ``: Implementation details.
   - ``: Verification commands (must include lint/truth.sh, lint/dp.sh, lint/task.sh).
3. CONSTRAINTS:
   - Stay strictly within plan scope.
   - All paths must exist in the dump or be explicitly planned creations.
   - Use storage/dp/active/allowlist.txt as the reference for file access (implied by content).

Output only: The full Slots File content enclosed in a markdown code block.

9.2 docs/ops/prompts/E-PROMPT-02.md (Hygiene)

Hygiene (Refresh + Conform to Slots)

Use when: Conforming an old DP or rough notes to the Phase 3 ops/bin/draft Slots format.

Attach: OPEN, dump, Old-DP.md (or rough draft).

Rules:

- Refresh state using the attached OPEN and dump artifacts.
- Logic: PoT.md. Structure: ops/bin/draft Slots Format.
- Output only the stance format.
- Preserve intent; update structure to match Phase 3 immutable workflow.

- **Path Syntax:** All file paths must be enclosed in backticks.

Steps:

1. NORMALIZE the input into ops/bin/draft Slot headers (, , etc.).
2. ENFORCE input discipline:
    - No citations of disposable artifacts (chat logs, scratch notes).
    - No pasted bundles in the body.
    - Concise, imperative language.
    - **Backticked Paths:** Ensure every file path is backticked.
3. VALIDATE allowlist:
    - Ensure all affected files are listed in [PLAN_CHANGELOG] to inform the allowlist sidecar.
    - Do not inline-expand large lists if not required by the slot, but ensure clarity.
4. VERIFY paths against the dump.

Output only: The full Slots File content enclosed in a markdown code block.

9.3 docs/ops/prompts/E-PROMPT-04.md (Analyst)

Analyst (Refresh + Discuss)

Use when: Reading the repo to answer questions (no edits).

Attach: OPEN, dump.

Rules:

- Refresh state using the attached OPEN and dump artifacts.
- Logic: PoT.md. Reference: docs/MAP.md and SoP.md.
- Read-only: no commands, no patches, no proposed file changes.
- **Minimize Operator Prose:** Structure the output to facilitate rapid decision-making.

Steps:

1. ANALYZE the Operator query using the attached context.
2. SYNTHESIZE findings based on PoT.md and repository state.
3. FORMULATE a 'Strategic Options' menu at the end of the analysis.

Output Structure:

1. Analysis/Discussion (The "Why" and "What").
2. **Strategic Options** (The "How"):
    - Present 2-3 distinct, actionable paths forward.
    - For each option, list **Pros**, **Cons**, and **Risk**.
    - Provide a **Recommendation** to guide the Operator.

Output: Discussion followed by the Strategic Options menu.

9.4 docs/ops/prompts/E-PROMPT-01.md (Gatekeeper)

Gatekeeper (Refresh + Audit)

Use when: Auditing worker output before merge.

Attach: DP-RESULTS.md, OPEN, OPEN-PORCELAIN (if any), dump payload, dump manifest (and dump bundle if any).

Rules:

- Refresh state using the attached OPEN and dump artifacts.
- Logic: PoT.md. Structure: TASK.md (Phase 3 Immutable). Output only the stance format.

Steps:

1. AUDIT for PoT.md compliance.
2. RECEIPTS: Confirm every changed file is accounted for by diffs/proofs in DP-RESULTS.md.
3. **ALLOWLIST ENFORCEMENT:**
    - Verify that storage/dp/active/allowlist.txt exists and is the sole source of truth for file access.
    - Confirm that every file changed in the PR is present in the sidecar allowlist.
    - Reject any changes to files not listed in the sidecar.
4. DRIFT CHECKS:
    - No out-of-scope or unauthorized changes.
    - **Context Hazard Check:** Ensure no library subtree (opt/_factory/) is included in global context unless explicitly authorized.
5. GENERATED OUTPUTS:
    - Confirm ops/bin/draft was used to generate the DP block (verify structure hash if available).
    - Reject manual edits to TASK.md structure.
    - Require tool-based regeneration for manifests/LLM bundles.

Output: PASS or FAIL. If FAIL, list specific deviations.

9.5 docs/ops/prompts/README.md

Stela Operator Prompts

Context:

- These prompts are for the Operator/Integrator running a session.

- Attach OPEN + dump to refresh repo state and confirm file paths.

Phase 3 Immutable Workflow Mandate:

- **Do not hand-edit TASK.md structural boilerplate.**
- Use E-PROMPT-03 (Architect) to generate a **Slots File**.
- Use the ops/bin/draft binary to generate the Dispatch Packet from the Slots File.
- Use storage/dp/active/allowlist.txt for file access control (Sidecar Allowlist).

Governance:

- Logic: PoT.md.
- Structure: TASK.md (enforced via ops/bin/draft).

Global rules (apply to all stances):

- Output only what the stance requests.
- Do not treat disposable artifacts (rough notes, chat logs, scratch drafts) as authoritative inputs.
- Do not paste OPEN/DUMP payloads or manifests into a DP body.
- Workers operate from the DP text and repo canon; OPEN and dump are for refresh and receipts.

10. Risk Analysis and Strategic Integration

The deployment of these prompt updates carries specific operational risks that must be managed.

Table 2: Risk Analysis of Prompt Deployment

Risk Vector	Impact	Mitigation Strategy
Parsing Mismatch	Agent generates invalid slots; ops/bin/draft fails.	Prompts mandate strict `` syntax. Hygiene prompt acts as a fallback sanitizer.
Backtick Hallucination	Agent fails to backtick paths, causing Markdown rendering issues.	Architect prompt explicitly lists "Backticked Paths" as a constraint.
Sidecar Desync	Agent updates the plan but forgets the allowlist sidecar logic.	Gatekeeper prompt explicitly audits the allowlist.txt against the [PLAN_CHANGELOG].

Operator Confusion	Operator attempts to paste Slots File directly into TASK.md.	README.md explicitly forbids hand-edits; ops/bin/draft is the only authorized write mechanism.

The "Strategic Options" menu in the Analyst prompt specifically mitigates the risk of **Decision Fatigue**. By pre-calculating the viable paths and their associated risks (e.g., "Option 1 violates PoT Axiom 1.2"), the Agent performs the heavy lifting of compliance checking before the Operator even makes a choice.

11. Conclusion

The refactoring of the Stela operator prompts is a foundational requirement for the success of Phase 3 Immutable Workflows. By transforming the Architect and Hygiene prompts into configuration generators for ops/bin/draft, the system effectively "compiles" intent into governance, removing the fragility of human-maintained Markdown. The simultaneous hardening of the Gatekeeper to enforce the allowlist.txt sidecar and the enhancement of the Analyst to provide "Strategic Options" creates a robust, self-correcting operational environment. These changes satisfy the core mandates of the Policy of Truth: eliminating ambiguity, enforcing physical laws (file access), and preventing drift through deterministic tooling.

The implementation of these prompts effectively closes the loop between the "Software" (AI Agents) and the "Hardware" (Bash scripts), ensuring that the Stela repository operates as a coherent, immutable system.

Works cited

1. dump-full-main-b7a7a2eb.txt